UNIVERSIDAD DE LA SALLE

Informe de pruebas de funcionalidad: aplicación de escritorio para cifrar y descifrar datos con Python y Tkinter

Nombre de la materia
Aplicaciones
Nombre del programa
Maestría en IA

Nombre del estudiante
Andres Felipe Cabeza Serrano
Código
1121952226

Nombre de la actividad
Aplicación de escritorio para cifrado y descifrado de datos
Unidad 2
Creación de interfaces gráficas de usuario (GUI) en Python

Nombre del profesor
Cristian Duney Bermúdez Quintero

Fecha
20 de octubre de 2026

INTRODUCCIÓN

Una organización necesita transmitir datos desde dispositivos móviles que podrían estar intervenidos. Para reducir el riesgo, los datos se cifran antes de enviarse y se descifran al recibirse. Esta actividad desarrolla una aplicación de escritorio en Python que realiza ese proceso sobre números enteros de seis dígitos, con una interfaz gráfica construida con Tkinter (Python Software Foundation, s. f.-a).

Los objetivos son (a) implementar el algoritmo de cifrado y descifrado indicado en el caso práctico, (b) construir tres ventanas (principal, cifrado y descifrado) con navegación entre ellas, (c) organizar el código en funciones y módulos reutilizables y (d) comprobar el funcionamiento con dos casos de prueba, 123456 y 791658. El código fuente y su historial de versiones están publicados en el repositorio del proyecto.

Repositorio: https://github.com/FelipeCabeza16/cifrado-python-mia

DESARROLLO

1. Algoritmo

El cifrado actúa sobre cada uno de los seis dígitos y luego sobre su posición. Primero, a cada dígito se le suma 7 y se toma el residuo de la división entre 10. Después se intercambian las posiciones del primer dígito con el tercero, del segundo con el cuarto y del quinto con el sexto.

El descifrado revierte los pasos en orden inverso. Como cada intercambio de posiciones es su propia inversa, se aplican de nuevo los mismos intercambios y después se resta 7 a cada dígito, también con residuo de la división entre 10. El número se maneja como texto y no como entero, para conservar los ceros a la izquierda (por ejemplo, 018932).

2. Estructura del código

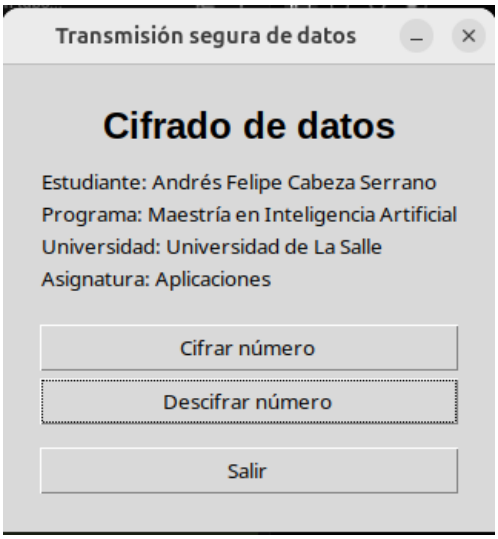
- logica.py: funciones validar_numero, intercambiar_posiciones, cifrar y descifrar. No depende de la interfaz, por lo que puede probarse y reutilizarse por separado. Las constantes LONGITUD, DESPLAZAMIENTO e INTERCAMBIOS evitan números sin nombre.
- ventana_principal.py: muestra los datos del autor y dos botones para abrir las demás ventanas.
- ventana_cifrado.py y ventana_descifrado.py: reciben el número, llaman a la función correspondiente y muestran el resultado; si la entrada no es válida, muestran un mensaje de error.
- main.py: punto de entrada que crea la ventana principal.
- tests/test_logica.py: pruebas unitarias con unittest (Python Software Foundation, s. f.-b).

La ventana principal es la raíz de la aplicación (Tk) y las otras dos son ventanas secundarias (Toplevel), de modo que se pueden abrir y cerrar sin terminar el programa. Se usaron nombres descriptivos en español, comentarios y docstrings en cada módulo y función.

3. Ventanas de la aplicación

Figura 1

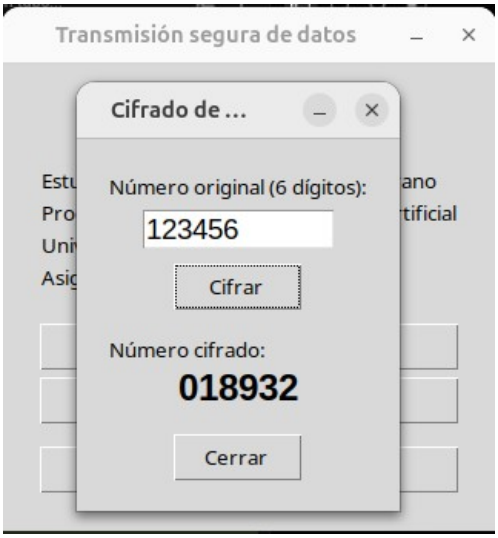
Ventana principal



Nota. Muestra los datos del autor y los botones Cifrar número, Descifrar número y Salir. Elaboración propia.

Figura 2

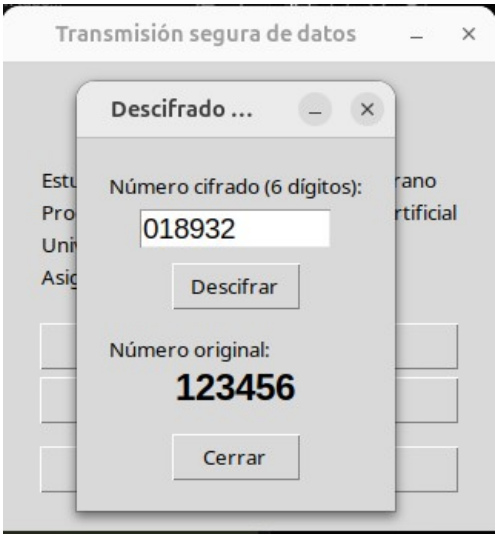
Ventana de cifrado



Nota. El usuario ingresa un número de 6 dígitos y la ventana muestra el número cifrado. Elaboración propia.

Figura 3

Ventana de descifrado

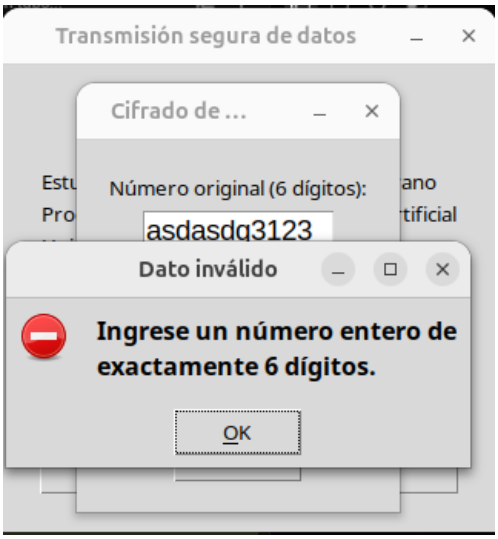


Nota. El usuario ingresa el número cifrado y la ventana muestra el número original. Elaboración propia.

Antes de cifrar o descifrar, la aplicación valida que la entrada tenga exactamente seis dígitos. Si el usuario escribe letras u otra cantidad de dígitos, se muestra un cuadro de diálogo y no se realiza ninguna operación.

Figura 4

Mensaje ante una entrada inválida



Nota. Se ingresó un texto con letras y la aplicación pidió un número de exactamente 6 dígitos. Elaboración propia.

PRUEBAS DE FUNCIONALIDAD

Se ejecutaron dos casos de prueba, cada uno en las dos direcciones: cifrar el número original y descifrar el resultado para recuperar el valor inicial. La Tabla 1 resume los pasos intermedios calculados a mano y los resultados mostrados por la aplicación.

Tabla 1

Resumen de los casos de prueba					
Caso	**Entrada**	**Tras sumar 7 (mod 10)**	**Tras intercambiar**	**Salida cifrada**	**Descifrado**
1	123456	8 9 0 1 2 3	0 1 8 9 3 2	018932	123456
2	791658	4 6 8 3 2 5	8 3 4 6 5 2	834652	791658

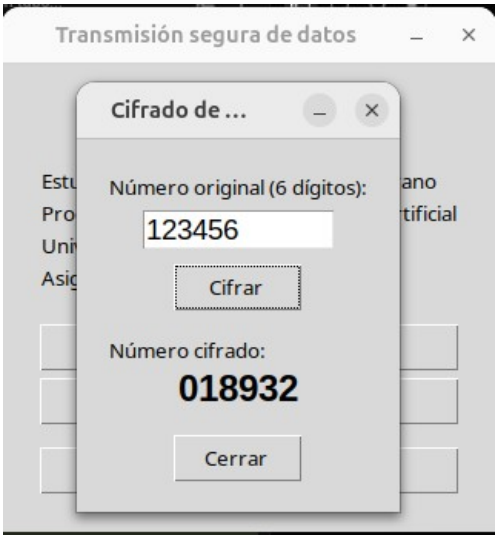
Nota. Los pasos intermedios se calcularon manualmente y coinciden con la salida de la aplicación. Elaboración propia.

Caso 1: número 123456

Al sumar 7 a cada dígito y tomar el residuo entre 10 se obtiene 8, 9, 0, 1, 2, 3. Al intercambiar el primero con el tercero queda 0, 9, 8, 1, 2, 3; el segundo con el cuarto, 0, 1, 8, 9, 2, 3; y el quinto con el sexto, 0, 1, 8, 9, 3, 2. El número cifrado es 018932, igual al que muestra la aplicación. Al descifrarlo se recupera 123456.

Figura 5

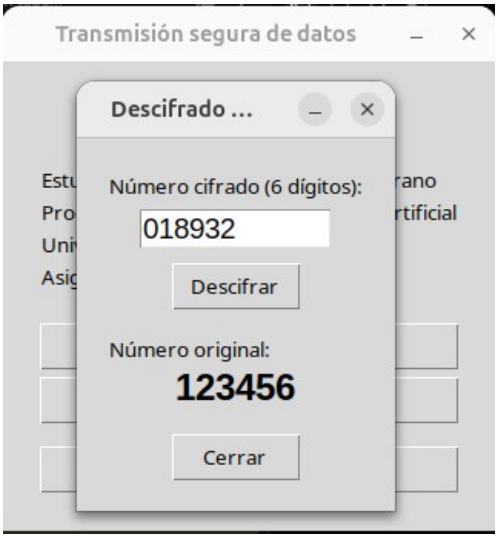
Caso 1: cifrado de 123456



Nota. Entrada 123456; salida 018932. Elaboración propia.

Figura 6

Caso 1: descifrado de 018932



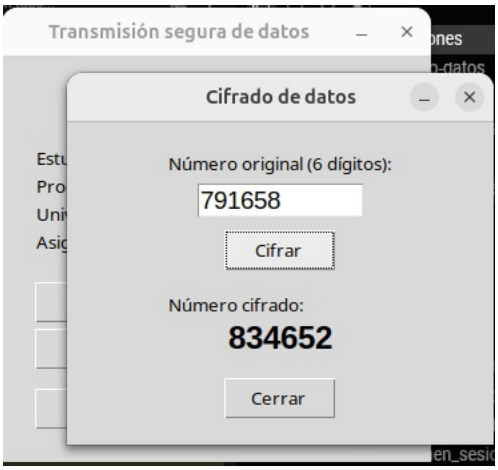
Nota. Entrada 018932; salida 123456. Elaboración propia.

Caso 2: número 791658

Al sumar 7 a cada dígito se obtiene 4, 6, 8, 3, 2, 5. Tras los tres intercambios queda 8, 3, 4, 6, 5, 2. El número cifrado es 834652, igual al que muestra la aplicación. Al descifrarlo se recupera 791658.

Figura 7

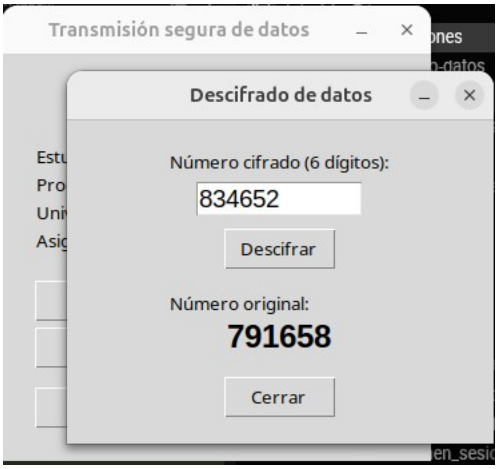
Caso 2: cifrado de 791658



Nota. Entrada 791658; salida 834652. Elaboración propia.

Figura 8

Caso 2: descifrado de 834652



Nota. Entrada 834652; salida 791658. Elaboración propia.

Pruebas automáticas

Además de las pruebas manuales en la interfaz, se escribieron cinco pruebas unitarias sobre la lógica: los dos casos anteriores, la conservación de ceros a la izquierda (000123), la ida y vuelta (descifrar(cifrar(n)) = n) sobre una muestra de números de todo el rango de 000000 a 999999 y el rechazo de entradas inválidas (vacías, de longitud incorrecta, con letras, espacios, signo o dígitos no ASCII). Todas se ejecutaron con éxito.

CONCLUSIONES

La aplicación cumple los requerimientos del caso práctico. Cifra y descifra correctamente los dos números de prueba, conserva los ceros a la izquierda y rechaza las entradas que no son de seis dígitos. Las tres ventanas se comunican entre sí y la navegación funciona como se esperaba.

Separar la lógica de la interfaz fue la decisión de diseño más útil: permitió probar el algoritmo sin abrir ventanas, reutilizar la función de intercambio de posiciones en ambos sentidos y mantener cada archivo con una sola responsabilidad. También se confirmó la importancia de validar los datos en el punto de entrada y de tratar el número como texto cuando los ceros iniciales son significativos.

Como limitación, el cifrado es una transformación sencilla (suma modular y permutación fija) sin clave secreta, por lo que sirve para fines didácticos y no ofrece la seguridad de algoritmos estándar como AES. Una mejora futura sería aceptar longitudes variables y usar una clave configurable.

REFERENCIAS

Bermúdez Quintero, C. D. (s. f.). *RED 6. Unidad 2.5. Creación de interfaces gráficas de usuario (GUI) en Python* [Recurso educativo digital]. Universidad de La Salle.

Python Software Foundation. (s. f.-a). *tkinter: Python interface to Tcl/Tk*. Python 3 documentation. https://docs.python.org/3/library/tkinter.html

Python Software Foundation. (s. f.-b). *unittest: Unit testing framework*. Python 3 documentation. https://docs.python.org/3/library/unittest.html